DEVELOPMENT DIARY

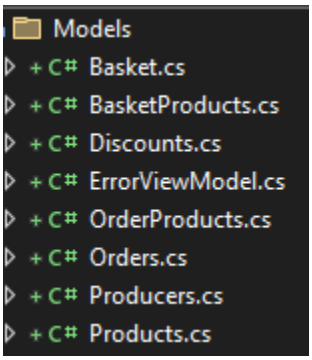
Nelson Ogodo

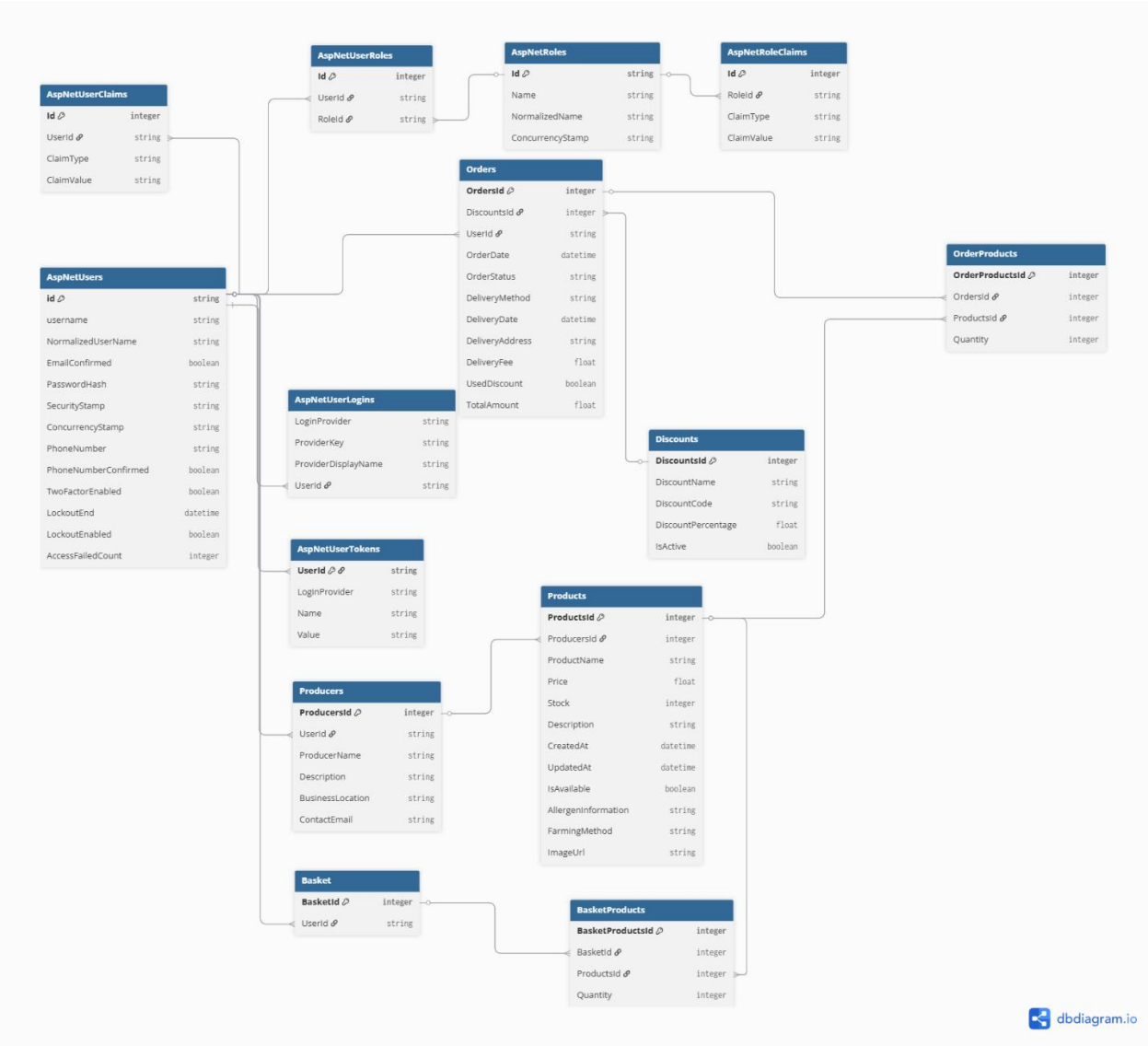
Contents

Day 1 1
Day 2 8
Day 3 13
Day 4 16
Day 5 18

Day 1

On this day I created and incorporated all of my models





I made some slight changes to my original models as I removed discount model as I deemed it unnecessary to the project prototype being created and instead added “DiscountApplid” property field to better incorporate the loyalty system. I also added “Status” and “BasketCreatedAt” field property which made it. I added Icollections and Navigation properties

```
namespace GreenFieldWeb.Models
{

    public class Basket
    {

        public int BasketId { get; set; }

        public string UserId { get; set; } // Links Each Basket to a User

        public bool Status { get; set; }
        9 references
        public DateTime BasketCreatedAt { get; set; } = DateTime.UtcNow;

        0 references
        public ICollection<BasketProducts>? BasketProducts { get; set; }
    }
}
```

```
namespace GreenFieldWeb.Models
{

    public class BasketProducts
    {

        public int BasketProductsId { get; set; }

        public int BasketId { get; set; }

        public int ProductsId { get; set; }

        public int Quantity {  get; set; }

        public Products Products { get; set; }
        9 references
        public Basket Basket { get; set; }
    }
}
```

```
namespace GreenFieldWeb.Models
{
    24 references
    public class OrderProducts
    {
        11 references
        public int OrderProductsId { get; set; }
        14 references
        public int OrdersId { get; set; }
        14 references
        public int ProductsId { get; set; }
        15 references
        public int Quantity { get; set; }
        9 references
        public Products Products { get; set; }
        14 references
        public Orders Orders { get; set; }

    }
}
```

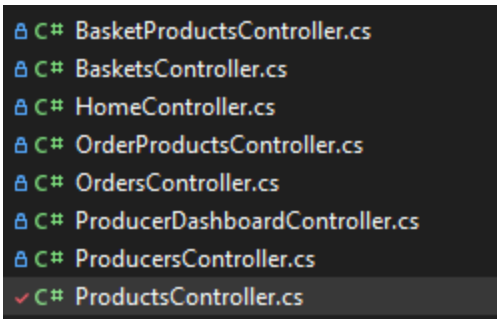
```
namespace GreenFieldWeb.Models
{
    public class Orders
    {
        public int OrdersId { get; set; }
        public string UserId { get; set; }
        public DateOnly OrderDate { get; set; }
        public string OrderStatus { get; set; }
        public string DeliveryMethod { get; set; }
        public DateOnly? DeliveryDate { get; set; }
        public string? DeliveryAddress { get; set; }
        public decimal DeliveryFee { get; set; }
        public decimal DiscountApplied { get; set; }
        public decimal TotalAmount { get; set; }

        public ICollection<OrderProducts>? OrderProducts { get; set; }
    }
}
```

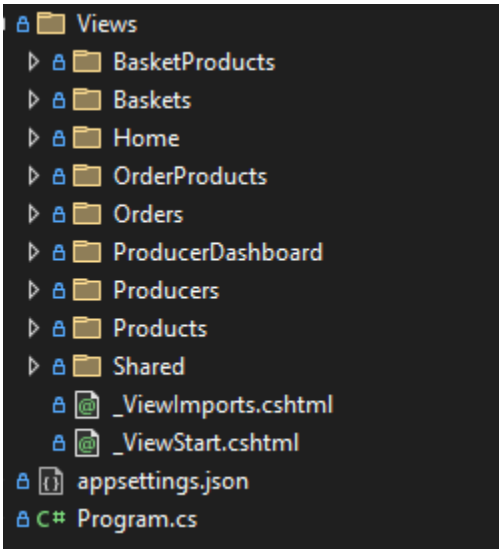
```
namespace GreenFieldWeb.Models
{

    public class Producers
    {

        public int ProducersId { get; set; }

        public string UserId { get; set; }

        public string ProducerName { get; set; }

        public string Description { get ; set; }

        public string BusinessLocation { get; set; }

        public string ContactEmail { get; set; }


        public ICollection<Products>? Products { get; set; }

    }
}
```

```
namespace GreenFieldWeb.Models
{

    public class Products
    {

        public int ProductsId { get; set; }

        public int ProducersId { get; set; }

        public string ProductName { get; set; }

        public decimal Price { get; set; }

        public int Stock {  get; set; }

        public string Description { get; set; }

        public DateTime CreatedAt { get; set; }

        public DateTime UpdatedAt { get; set; }

        public bool IsAvailable { get; set; }

        public string AllergenInformation { get; set; }

        public string FarmingMethod { get; set; }

        public string? ImageUrl { get; set; }

        public Producers Producers { get; set; }

        0 references
        public ICollection<OrderProducts>? OrderProducts { get; set; }
        0 references
        public ICollection<BasketProducts>? BasketProducts { get; set; }

    }
}
```

I also scaffolded all my Models to produce comprehensive controllers and views for each Model. I also created a ProducerDashboard controller and view to incorporate the Producer Dashboard feature.

I also added Migration so it generates a file describing what changeas to be made and then I “Update-Database so it then applies those changes to the actual database.



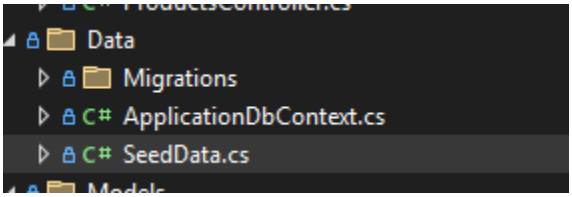


At the end of Day 1

- Created the project using ASP.NET CORE
- Added Migration and Update-Database
- Created models and scaffolded each model
- Created controllers and views for Producer Dashboard

Day 2

I created a SeedData class in the Data folder to systematically populate the database with test data and startup. I did this because manually re-entering data every time the database resets would be time consuming. The seeding runs through program.cs and it only inserts data if it doesn't already exist so it never duplicates. I seeded 4 users and assigned roles to each of them so each role can control what each user can access across the site. After the users were created I seeded the producers table linking each producer to its user account via UserId. I then seeded Products last as they require producers to already exist in the database due to the ProducersId foreign key.



```
1 references
public class SeedData
{

    1 reference
    public static async Task SeedUsersAndRoles(IServiceProvider serviceProvider, UserManager<IdentityUser> userManager, RoleManager<IdentityR
    {//Seeded Roles
        string[] roleNames = { "Admin", "Producer", "Standard", "Developer" };
        foreach (string roleName in roleNames)
        {
            var roleExists = await roleManager.RoleExistsAsync(roleName);
            if (!roleExists)
            {
                var role = new IdentityRole(roleName);
                await roleManager.CreateAsync(role);
            }

        }
```

```
var adminUser = await userManager.FindByEmailAsync("admin@example.com");
if (adminUser == null)
{
    adminUser = new IdentityUser { UserName = "admin@example.com", Email = "admin@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(adminUser, "Password123!");

}
if (!await userManager.IsInRoleAsync(adminUser, "Admin"))
{
    await userManager.AddToRoleAsync(adminUser, "Admin");
}
var producerUser = await userManager.FindByEmailAsync("producer@example.com");
if (producerUser == null)
{
    producerUser = new IdentityUser { UserName = "producer@example.com", Email = "producer@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(producerUser, "Password123!");

}
if (!await userManager.IsInRoleAsync(producerUser, "Producer"))
{
    await userManager.AddToRoleAsync(producerUser, "Producer");
}
var producerUser2 = await userManager.FindByEmailAsync("producer2@example.com");
if (producerUser2 == null)
{
    producerUser2 = new IdentityUser { UserName = "producer2@example.com", Email = "producer2@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(producerUser2, "Password123!");
}
if (!await userManager.IsInRoleAsync(producerUser2, "Producer"))
{
    await userManager.AddToRoleAsync(producerUser2, "Producer");
}
var producerUser3 = await userManager.FindByEmailAsync("producer3@example.com");
if (producerUser3 == null)
{
    producerUser3 = new IdentityUser { UserName = "producer3@example.com", Email = "producer3@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(producerUser3, "Password123!");
}
if (!await userManager.IsInRoleAsync(producerUser3, "Producer"))
```

```
}
var devUser = await userManager.FindByEmailAsync("dev@example.com");
if (devUser == null)
{
    devUser = new IdentityUser { UserName = "dev@example.com", Email = "dev@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(devUser, "Password123!");
}
if (!await userManager.IsInRoleAsync(devUser, "Developer"))
{
    await userManager.AddToRoleAsync(devUser, "Developer");
}
var normalUser = await userManager.FindByEmailAsync("user@example.com");
if (normalUser == null)
{
    normalUser = new IdentityUser { UserName = "user@example.com", Email = "user@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(normalUser, "Password123!");
}
if (!await userManager.IsInRoleAsync(normalUser, "Standard"))
{
    await userManager.AddToRoleAsync(normalUser, "Standard");
}
```

```
var context = serviceProvider.GetRequiredService<ApplicationDbContext>();
//Finds User by Email
var ProducerUser1 = await userManager.FindByEmailAsync("producer@example.com");
var ProducerUser2 = await userManager.FindByEmailAsync("producer2@example.com");
var ProducerUser3 = await userManager.FindByEmailAsync("producer3@example.com");

if (ProducerUser1 == null || ProducerUser2 == null || ProducerUser3 == null)
{
    throw new Exception("Producer users not found. Ensure they are seeded before running this method.");
}
if (context.Producers.Any())
    return;

var producers = new List<Producers>
{
    new Producers
    {
        ProducerName = "Fresh Fields",
        Description = "Fresh Fields is a farm that has been providing high-quality produce for over 20 years. ",
        ContactEmail = "contact@freshfields.co.uk",
        BusinessLocation = "Costwolds",
        UserId = ProducerUser1.Id
    },
    new Producers
    {
        ProducerName = "Green Acres",
        Description = "Green Acres is a family-owned farm that specializes in organic produce. ",
        ContactEmail = "contact@greenacres.co.uk",
        BusinessLocation = "Yorkshire",
        UserId = ProducerUser2.Id
    },
    new Producers
    {
        ProducerName = "Sunny Farms",
        Description = "Sunny Farms is a farm that focuses on sustainable farming practices and offers a wide variety
        ContactEmail = "contact@sunnyfarms.co.uk",
        BusinessLocation = "Devon",
        UserId = ProducerUser3.Id
    }
};
context.Producers.AddRange(producers);
```

```
var context = serviceProvider.GetRequiredService<ApplicationDbContext>();
var FreshFields = await context.Producers.FirstOrDefaultAsync(s => s.ProducerName == "Fresh Fields");
var GreenAcres = await context.Producers.FirstOrDefaultAsync(s => s.ProducerName == "Green Acres");
var SunnyFarms = await context.Producers.FirstOrDefaultAsync(s => s.ProducerName == "Sunny Farms");
if (FreshFields == null || GreenAcres == null || SunnyFarms == null)
{
    throw new Exception("Producers not found. Ensure they are seeded before running this method.");
}

if (context.Products.Any())
    return;
var products = new List<Products>
{
    new Products
    {
        ProductName = "Organic Honey",

        Price = 1.88m,
        Stock = 50,
        Description = "Pure organic honey collected from wildflowers.",
        CreatedAt = new DateTime(2026, 1, 1),
        UpdatedAt = new DateTime(2026, 1, 10),
        IsAvailable = true,
        AllergenInformation = "None",
        FarmingMethod = "Organic",
        ProducersId = FreshFields.ProducersId,
        ImageUrl = "/images/organic-honey.jpg"


    },
    new Products
    {
        ProductName = "Free-range Eggs",
        Price = 5.99m,
        Stock = 200,
        Description = "Eggs from free-range chickens.",
        CreatedAt = new DateTime(2025, 3, 1),
        UpdatedAt = new DateTime(2025, 3, 3),
        IsAvailable = true,
        AllergenInformation = "Contains Eggs",
        FarmingMethod = "Free-range",
```

```
        ImageUrl = "/images/free-range-pork.jpg"
    },
    new Products
    {
        ProductName = "Fresh Blueberries",
        Price = 3.49m,
        Stock = 90,
        Description = "Sweet and plump fresh blueberries.",
        CreatedAt = new DateTime(2026, 4, 1),
        UpdatedAt = new DateTime(2026, 4, 10),
        IsAvailable = true,
        AllergenInformation = "None",
        FarmingMethod = "Conventional",
        ProducersId = SunnyFarms.ProducersId,
        ImageUrl = "/images/fresh-blueberries.jpg"
    },
    new Products
    {
        ProductName = "Organic Potatoes",
        Price = 0.79m,
        Stock = 200,
        Description = "Starchy and delicious organic potatoes.",
        CreatedAt = new DateTime(2025, 8, 1),
        UpdatedAt = new DateTime(2025, 8, 10),
        IsAvailable = true,
        AllergenInformation = "None",
        FarmingMethod = "Organic",
        ProducersId = FreshFields.ProducersId,
        ImageUrl = "/images/organic-potatoes.jpg"
    }

};
context.Products.AddRange(products);
await context.SaveChangesAsync();
```

I encountered an error because my code was supposed to check if the admin user already existed and create them if not but the condition was written backwards so it did the opposite and tried tocreate a user if it already exists which meant it only tried to create a user that was already there, and skipped creation when thew user was actually missing. So when the code later tried to check the roles of the user, the user was null and the app crashed since you cant check roles od someone who doesn't exist

So to fix this i flipped the condition from if(adminUser != null) to if (adminUser == null) so itll create a user if it doesn't exist.

```
var adminUser = await userManager.FindByEmailAsync("admin@example.com");
if (adminUser == null)
{
    adminUser = new IdentityUser { UserName = "admin@example.com", Email = "admin@example.com", EmailConfirmed = true };
    await userManager.CreateAsync(adminUser, "Password123!");
}
if (!await userManager.IsInRoleAsync(adminUser, "Admin"))
{
    await userManager.AddToRoleAsync(adminUser, "Admin");
}
```

I encountered another error that is really similar to the previous one. My code was supposed to check if a role existed and if it didn't then it would be created. But like before it was written backwards which meant it was trying to do the opposite. So when the code later tried to assign the "Admin" role to a user, that role didn't exist in the database causing the app to crash because you cant assign a role that was never created. To fix this I Flipped it so if the role didn't exist it would be created.

```
if (!await roleManager.RoleExistsAsync(roleName))
{
    var role = new IdentityRole(roleName);
    await roleManager.CreateAsync(role);
}
```

I encountered a CSO246 error stating that the type "Producers" could not be found. This error occurred inside the Seed Data class when it tried to create and seed producer records into the database. The cause was a missing "using" directive a the top of the file. So to fix this I added this to the top of the Seed data class. The application built and ran successfully after this change.

```
using GreenFieldWeb.Models;
```

At the end of this day

- Seeded Users and User Roles
- Seeded Products
- Seeded Producers
- Fixed errors regarding Seed Data

Plan for tomorrow

- Work on Products Controller logic
- Work on Basket Controller logic
- Work on the style sheet code

Day 3

Today I worked on my CSS. The styling fir the Greenfield Local Hub website was handled through site.css file located in the wwroot/css/. I'm doing this rather than writing separate CSS for each page so all

colors, spacing and typography are defined ass CSS variables at the top of the file and reused throughout every view. CSS custom variables are declared inside :root which makes them global across every page

```
:root {
    --bg-body:        #FAF7F2;
    --bg-card:        #FFFFFF;
    --bg-subtle:      #F0EDE8;
    --text-primary:   #2B2B2B;
    --text-muted:     #6B6B6B;
    --green-dark:     #1A4A1A;
    --green-mid:      #2F6F4E;
    --green-light:    #6BBF59;
    --green-accent:   #639922;
    --loyalty:        #F4A259;
    --success:        #4CAF50;
    --warning:        #E9B44C;
    --error:          #D9534F;
    --border:         #E0DDD5;
    --radius:         30px;
    --radius-md:      16px;
    --radius-sm:      10px;
    --shadow:         0 4px 16px rgba(0,0,0,0.07);
    --shadow-lg:      0 12px 40px rgba(0,0,0,0.12);
    --transition:     0.2s ease;
    --font-size:      15px;
```

After that I worked on my basket logic. I added a button for users to be able to add to basket and then worked on the backend to allow the user to be able to add a product to the basket. I then improved on the basket page to allow the basket to display the items in the basket, the cost of each item in the basket, the quantity of each item in the basket, the option to remove from basket and the option to checkout. I then worked on the loyalty discount calculation logic in the controller. I also noticed that when I clicked the add to basket button it redirects immediately to the basket page so I added Ajax so the function can run silently in the background without redirecting to the basket page

```
decimal discount = 0m;

if (orderCount >= 5)
{
    discount = subtotal * 0.10m;
}

decimal total = subtotal - discount;

ViewBag.TotalAmount = subtotal;
ViewBag.Discount = discount;
ViewBag.Total = total;
ViewBag.OrderCount = orderCount;

return View(basketProducts);
```

I then encountered an error when trying to add a product to basket. And there were two bugs causing this. Firstly, the original scaffolded index method was returning the entire basket table. The view however was designed to receive Basket Products including products names and prices, so passing a list of Basket objects caused a model type mismatch error when running the program. The other bug was that the return statement was in the if (basket == null) block which meant it only returned data when a new basket was being created so if the user already had an existing basket the method had no return statement and crashed when running the program. So to fix this the basket products query and return statement of basket products were moved outside the if block so they always execute whether the basket was already created. The return type was also corrected to pass basket products instead of the basket table.

An unhandled exception occurred while processing the request.

InvalidOperationException: The model item passed into the ViewDataDictionary is of type 'System.Collections.Generic.List`1[GreenFieldWeb.Models.Basket]', but this ViewDataDictionary instance requires a model item of type 'System.Collections.Generic.IEnumerable`1[GreenFieldWeb.Models.BasketProducts]'.

Microsoft.AspNetCore.Mvc.ViewFeatures.ViewDataDictionary.EnsureCompatible(object value)

Stack Query Cookies Headers Routing

InvalidOperationException: The model item passed into the ViewDataDictionary is of type 'System.Collections.Generic.List`1[GreenFieldWeb.Models.Basket]', but this ViewDataDictionary instance require a model item of type 'System.Collections.Generic.IEnumerable`1[GreenFieldWeb.Models.BasketProducts]'.

I also ran into an issue in my basket with .ToString("0.00") appearing in the basket view. So I fixed the code by putting it in the razor expression.

I then worked on the checkout functionality, allowing the user to navigate to the checkout once clicking the checkout button then prompts the user to choose between delivery or checkout and depending on the user's decision the system will allow them to enter their address and choose a pickup date. I also added validation for the delivery address and collection Date. I also worked on the Orders controller to allow the checkout functionality to work

At the end of this day I had:

- Finished Basket functionality
- Created and improved the Products Controller
- Fixed Errors
- Made Checkout functionality

Tomorrow I want to:

- Complete Order Page
- Continue working on the styles sheet of the solution
- Make the Producer Dashboard

Day 4

Today I worked on the orders controller. First, I included a step that allowed the system to find currently logged in users and assign values for the order of the user. Then I set the order date and the Order status to be pending by default. Then I worked on making sure the system found all the items in the

basket. Then I worked on the subtotal calculation and the discount calculator. I also made the system handle delivery or collections. I also added error checks then I finished the orders controller logic. I then edited the order index so it could show orders and then once the details button is clicked it shows a breakdown of the products in the orders. I also added a feature to restrict users from seeing any other user's order history.

I also ran into an error because the property was called "Subtotal" instead of TotalAmount so it didn't match my model

```
<div>
    <span>Subtotal:</span>
    <span>£@(((decimal)ViewBag.Subtotal).ToString("0.00"))</span>
</div>
```

I fixed it by changing it to Total Amount

```
var orderProduct = new OrderProducts
{
    OrdersId = orders.OrdersId,
    ProductsId = basketProduct.ProductsId,
    Quantity = basketProduct.Quantity
};

_context.OrderProducts.Add(orderProduct);
basketProduct.Products.Stock -= basketProduct.Quantity;

// Close basket
basket.Status = false;
await _context.SaveChangesAsync();

return RedirectToAction("Index", "Home")
```

```
public async Task<IActionResult> Create([Bind("OrdersId,OrderDate,OrderStatus,DeliveryMethod,DeliveryDate,DeliveryAddress,DeliveryFee,DiscountApplied,TotalAmount")] Orders orders, int basketId)
```

I fixed this by removing the return Redirect to Action("Index,"Home) statement out of the for loop which thus allows the method to returns a value.

I then worked on my Producer Dashboard functionality with the already made controller and views. On the controller I put the Authorize code on the entire controller just for the Producer, developer, and admin role. I then added the context to the controller.

```
[Authorize(Roles = "Producer,Developer,Admin")]
1 reference
public class ProducerDashboardController : Controller
{
    private readonly ApplicationDbContext _context;

    0 references
    public ProducerDashboardController(ApplicationDbContext context)
    {
        _context = context;
    }
    0 references
```

I then edited the index method and the index view to enable it to display the Producer Products and enable them to edit and update orders. I then added features that allowed Producers to manage stock levels and view all their products, delete products, add products. After that I added a feature to all Producers to Update the tracking status of orders. After that I then continued working on my CSS for the solution. I then added images to my products

At the end of the day I had:

- Completed Orders page functionality
- Worked on CSS
- Finished Producer Dashboard functionality
- Added images to products page

Tomorrow I want to:

- Integrate a GMAIL log in system
- Add accessibility features
- Add authorization

Day 5

Today I integrated the GMAIL log in system using google developer console.

OAuth 2.0 Client IDs

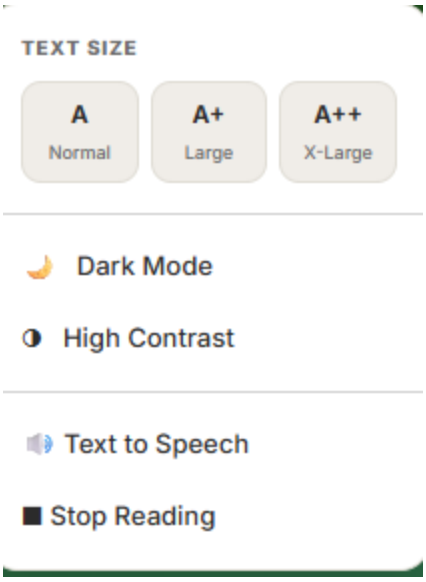
Name	Creation date	Type	Client ID
GreenField Local Hub Client	23 Apr 2026	Web application	533146252326-djg6...

Then I added my client Id and client secret to the appsettings.json file.

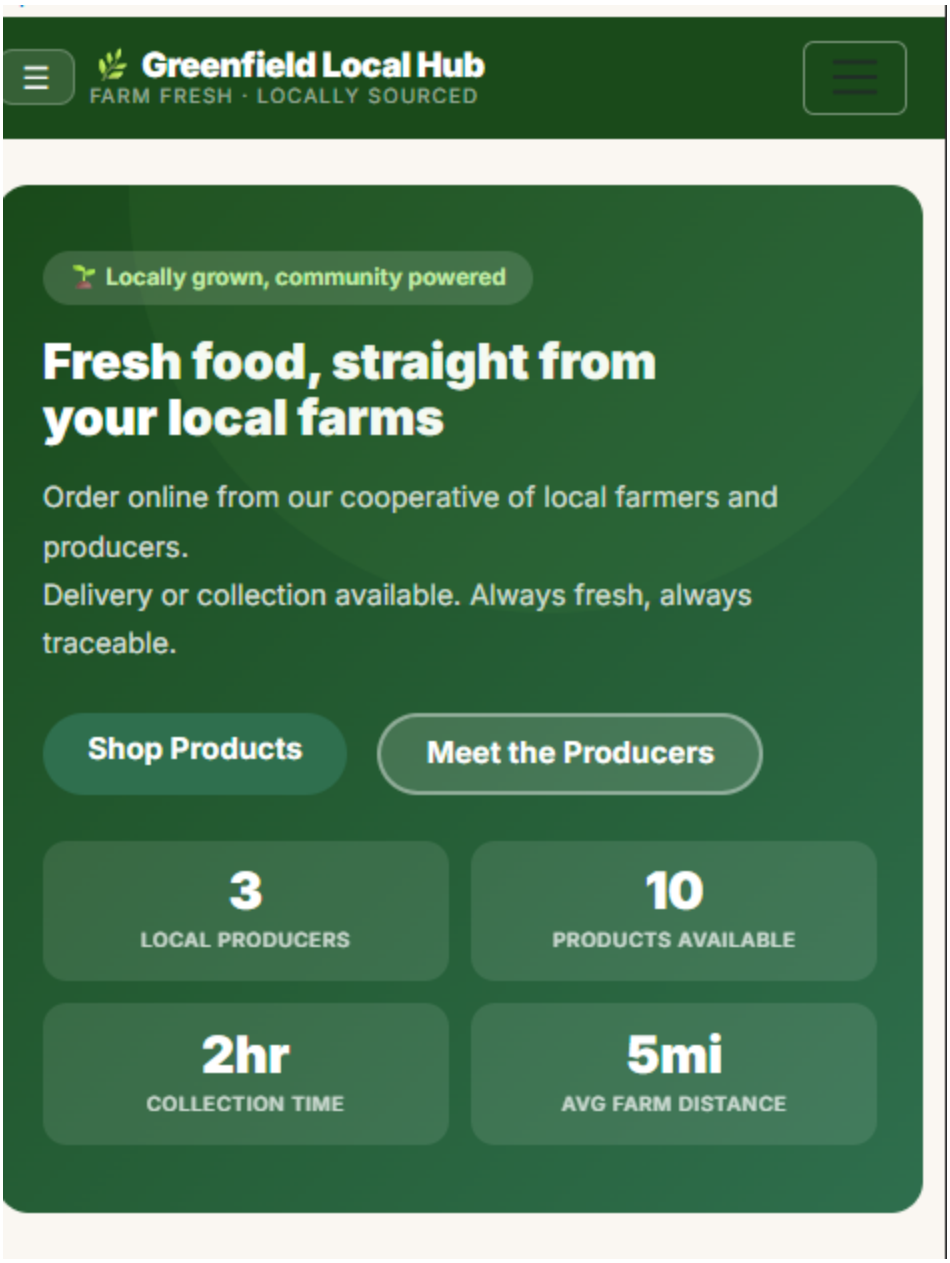
```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=aspnet-GreenFieldWeb-18ad65
  },
  "Google": {
    "ClientId": "533146252326-djg6m638mmrt8t58bn39jppiebhl2ja5.apps.googleusercontent.com",
    "ClientSecret": "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"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Overall the integration was functional. I then added authorization to every page.

Then I added accessibility features to the solution such as Screen reader, Text stings, high contrast mode, dark mode and light mode. For the dark mode feature I implemented it using data-time attribute on the html element. So when the user clicks the dark mode toggle in the accessibility menu, java script sets data-theme=”dark” on the page and then saves the preference to the local storage so it remains active between sessions. For the High contrast mode I had a similar approach to it using te data-contrast=”high” which sets black backgrounds and white text to meet WCAG 2.1 AA contrast requirements for users with visual impairment.



I also incorporated the NavBar using Bootstrap’s collapse system for the hamburger menu on mobile screens.



The basket button is styled as a green pill to draw the user’s attention to it.